

23CSE301 MACHINE LEARNING · REVIEW 1

How the regression notebook works

A cluster-by-cluster, cell-by-cell reading of
`notebooks/regression.ipynb`

NYC Taxi Trip Duration · 352,000 trips · ten algorithms
Every later cluster uses the table produced by the cluster above it.

Keep this beside the notebook during the viva.

Cell numbers match notebooks/regression.ipynb · Run All order is top to bottom.

Contents

- 01 How to read this guide
- 02 The data, in one page
- 03 A — Setup
- 04 B — Problem and data audit
- 05 C — Exploratory data analysis
- 06 D — Data cleaning
- 07 E — Feature engineering
- 08 F — Split, encode, scale
- 09 G — Ten regression models
- 10 H — Compare, tune, check
- 11 I — Manual tester
- 12 J — PDF report
- 13 Viva cheat-sheet

How to read this guide

The notebook is not a script you run from line 1 of a .py file. It is 90 cells in ten clusters (A–J). Markdown cells are the spoken explanation. Code cells are what Python executes. This PDF walks the same order as Run All.

For every code cell you get: (1) the purpose, (2) the source in a cream box, (3) a table that maps line groups to meaning. Plotting boilerplate (`set_title`, `set_xlabel`, `finalise`) is grouped so the page stays readable — those lines only label axes and save a PNG into `results/figures/`.

Golden rule. Variables created in an earlier cell stay in memory. `df_raw` from Cluster B is still there in Cluster J. If you skip a cluster, later cells crash.

Two names you must not confuse. `progress()` is the timestamp logger so Run All is not silent. Inside cleaning, a nested function also called `log()` only appends a row to the cleaning report — it is not the same helper.

The data, in one page

File: `data/nyc_taxi_trip_duration.csv` — 352,000 yellow-cab trips (Jan–Jun 2016), sampled from the original 1.46M-row table with seed 42.

#	Code	What it does
1	<code>id</code>	Unique trip label. Dropped. Not a predictor.
2	<code>vendor_id</code>	Which company logged the trip (1 or 2). Kept, one-hot encoded.
3	<code>pickup_datetime</code>	When the meter started. Parsed into hour / weekday / month / rush-hour, then dropped.
4	<code>dropoff_datetime</code>	When the passenger got out. Dropped — duration is dropoff minus pickup (leakage).
5	<code>passenger_count</code>	People in the cab. Keep 1–6.
6	<code>pickup_longitude / latitude</code>	Where the trip started (WGS84).
7	<code>dropoff_longitude / latitude</code>	Where the trip ended. Known as the destination, not as dropoff time.
8	<code>store_and_fwd_flag</code>	N or Y. Y = stored on the device and sent later. Rare. One-hot encoded.
9	<code>trip_duration</code>	TARGET. Seconds the trip lasted. This is y.

Engineered later (Cluster E): `trip_distance_km` (straight-line), `manhattan_distance_km` (grid), `bearing_deg` (compass), `pickup_hour / dayofweek / month, is_weekend, is_rush_hour`.

A

Setup

No modelling yet. Load libraries, lock the random seed, set folders.

Takes in
Nothing from the CSV.

Produces
Imported names, RANDOM_STATE=42,
RESULTS_DIR, progress().

Markdown cells 0–2 are the title, the cluster map, and the note that Run All writes into the currently executing cell — not a terminal at the top of the file.

Cell 3 · code · Imports, plot style, progress logger

```
from pathlib import Path
import sys, time, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from sklearn... (models, split, metrics, ColumnTransformer)
from IPython.display import display
from matplotlib.backends.backend_pdf import PdfPages
...
def progress(step, detail=''): print timestamped line, flush=True
```

#	Code	What it does
1–4	Path, sys, time, warnings	Stdlib: file paths, unbuffered prints, clocks, warning filter.
6–9	numpy, pandas, pyplot, seaborn	Arrays, tables, figures. Seaborn sits on matplotlib.
11–18	train_test_split, Grid/RandomizedSearchCV, cross_val_score	Split, hyperparameter search, 5-fold CV — used in F and H.
19–20	StandardScaler, OneHotEncoder, PolynomialFeatures	Scale numbers, dummy-code categories, expand interactions (G5).
21–22	ColumnTransformer, Pipeline	Bundle scaler+encoder as one object; chain poly □ linear.
23	LinearRegression, Ridge, Lasso, ElasticNet	The four linear / regularised models.
24–29	DecisionTree, RandomForest, GBM, HistGBM	Tree family. HistGBM is the sklearn booster we actually fit.
30–32	SVR, KNeighborsRegressor	Kernel and neighbour models (trained on subsets).
33	r2_score, MAE, root_mean_squared_error	The three mandatory test metrics.
34	TransformedTargetRegressor	Scale y for SVR, then invert predictions back to seconds.
35–36	display, PdfPages	Pretty tables in Jupyter; multi-page PDF in Cluster J.
38–40	warnings / pandas display options	Hide FutureWarning; show 50 columns; 4 decimal places.
42–43	RANDOM_STATE = 42; np.random.seed(42)	Reproducible splits, samples, and model shuffles.
45–53	sns.set_theme + plt.rcParams	Colorblind palette, default figure size, labelled axes.
55–57	print library versions	So a re-run can be matched to sklearn 1.6 / pandas 2.3.
59–64	stdout/stderr line_buffering	Without this, Run All looks frozen until a cell ends.
67–73	def progress(...)	Prints [HH:MM:SS] step detail to stdout and stderr, flushed.
75	progress('Cluster A', ...)	First visible heartbeat that setup finished.

Cell 4 · code · Knobs: paths, cleaning bounds, subset sizes

#	Code	What it does
6-7	<code>FAST_DEV = False</code>	False = train linear/tree/boosting on all cleaned rows. True = 120k-row practice run.
8	<code>WORKING_SAMPLE_SIZE = 120_000</code>	Only used when FAST_DEV is True (Cluster E).
10	<code>PLOT_SAMPLE = 25_000</code>	How many points EDA scatters draw. Stats still use the full table.
11	<code>TEST_SIZE = 0.20</code>	20% hold-out. Training gets 80%.
12	<code>EVAL_TEST_SIZE = 50_000</code>	If the 20% test set is bigger than this, everyone is scored on the same 50k slice.
15-16	<code>NYC_LONG/LAT min/max</code>	Bounding box for real NYC GPS. Outside = junk.
17	<code>MIN/MAX_DURATION_S = 60, 7200</code>	Keep 1 minute–2 hours. Drop cancelled and multi-hour errors.
18	<code>MIN/MAX_PASSENGERS = 1, 6</code>	0 is invalid; 7+ is not a yellow cab.
21-23	<code>SVR_TRAIN_SIZE, KNN_TRAIN_SIZE, TUNE_SAMPLE_SIZE</code>	12k / 40k / 40k. SVR is slow to fit; KNN is slow to predict; forest search uses a subset then refits.
25-27	<code>DATA_PATH</code>	CSV relative to the notebook folder, with a fallback if you run from the repo root.
29-33	<code>RESULTS_DIR, FIG_DIR, SAVED_FIGURES</code>	Create results/ and results/figures/. <code>finalise()</code> appends PNG paths here for Cluster J.
35-38	<code>print + progress</code>	Confirm FAST_DEV, absolute data path, results folder.

B**Problem and data audit**

Load the CSV once. Report shape, types, missingness, target distribution.

Takes in
data/nyc_taxi_trip_duration.csv

Produces
df_raw — 352,000 × 11, untouched.

Markdown cells 5–6, 8 state the prediction task (seconds) and that a mean predictor has $R^2 = 0$.

Cell 7 · code · Read the file

#	Code	What it does
1	<code>progress(... loading)</code>	Announce Cluster B in the Run All log.
2	<code>df_raw = pd.read_csv(DATA_PATH)</code>	The only read of the raw CSV. Every later cluster starts from this object (or a copy).
3	<code>print shape</code>	Rows × columns — rubric A1.
4	<code>df_raw.head()</code>	Show the first five trips so columns are visible.

Cell 9 · code · Audit table

#	Code	What it does
1–10	<code>audit = DataFrame({dtype, non_null, missing, missing_pct, n_unique})</code>	One row per column. This file should show missing = 0 everywhere.
11	<code>audit</code>	Display it. n_unique on id should equal the number of rows.

Cell 10 · code · Duplicates, counts, target describe

#	Code	What it does
1–2	<code>duplicated()</code> on rows and on id	Exact duplicate trips / reused ids. Both should be 0.
4–13	<code>value_counts</code> for vendor, flag, passenger_count	Imbalance: mostly 1 passenger, flag almost always N, two vendors.
15–16	<code>trip_duration.describe(percentiles=...)</code>	Mean vs median (skew), 1% and 99% tails, max (impossible if ~24 h).
17–19	<code>min/max pickup_datetime</code>	Confirms Jan–Jun 2016 coverage.

C

Exploratory data analysis

See the raw table before cleaning. Plots use 25k points; printed numbers use all rows.

Takes in
df_raw

Produces
Charts + eda (datetime parsed) + haversine preview.
Nothing is deleted yet.

Cell 13 · code · Helpers + EDA copy

#	Code	What it does
1	<code>progress(...)</code>	Log that EDA started.
2-5	<code>take_plot_sample(frame, n)</code>	If the table is larger than n, draw a random n rows with seed 42. Otherwise copy all.
8-19	<code>finalise(fig, name)</code>	tight_layout; take the figure; slug the title into a filename; save PNG into FIG_DIR; append to SAVED_FIGURES; plt.show().
22	<code>eda = df_raw.copy()</code>	Work on a copy so df_raw stays raw for Cluster D.
23	<code>to_datetime(pickup_datetime)</code>	Strings become real timestamps.
24-25	<code>dt.hour, dt.dayofweek</code>	Preview calendar features before we officially engineer them in E.
26	<code>loglp(trip_duration)</code>	log(1+seconds) — only for a nicer histogram, not for training.
27-28	<code>plot_df = take_plot_sample(eda)</code>	25k-row drawing sample.

Cell 14 · code · Target histograms

#	Code	What it does
2	<code>subplots(1, 2)</code>	Two panels side by side.
4-7	<code>histplot raw seconds</code>	Left: most mass at 5–15 min, long right tail.
9-12	<code>histplot loglp</code>	Right: more mound-shaped. We still predict raw seconds.
14	<code>finalise(fig)</code>	Save + show.
15-21	<code>print skew, median, mean, 99th pct</code>	Numbers from the full eda table, not just the 25k sample.

Cell 16 · code · Passenger, vendor, hour, weekday

#	Code	What it does
2	<code>subplots(2, 2)</code>	Four small charts.
4-7	<code>countplot passenger_count</code>	Solo riders dominate.
9-12	<code>countplot vendor_id</code>	Two vendors, roughly balanced.
14-17	<code>histplot pickup_hour, 24 bins</code>	Diurnal demand.
19-25	<code>countplot dayofweek + Mon-Sun labels</code>	Weekend vs weekday mix.
27	<code>finalise(fig)</code>	Save + show.

Cell 18 · code · Pickup / dropoff maps

#	Code	What it does
2	<code>subplots(1, 2)</code>	Pickup left, dropoff right.
4-12 / 14-24	<code>scatterplot lon vs lat</code>	s=8, alpha=0.25 so 25k points do not become a black blob.
26-28	<code>print min/max lon/lat on full eda</code>	If the range hits California, GPS junk exists — Cluster D will clip it.

Cell 20 · code · Haversine preview + distance/hour vs duration

#	Code	What it does
2-8	<code>def haversine_km(...)</code>	Great-circle km on a sphere of radius 6371 km. Used again in Cluster E.
11-16	<code>eda['trip_distance_km'] = haversine_km(...).to_numpy()</code>	Vectorised over all rows.
19-32	<code>scatter distance vs duration, xlim 0-40 km, ylim 0-8000 s</code>	Hide absurd axes so the real cloud is visible.
34-42	<code>boxplot duration by hour, showfliers=False</code>	Congestion cycle without 24-hour outliers stealing the scale.
46-47	<code>Pearson corr(distance, duration) on full eda</code>	Weak on RAW data because of those outliers; jumps after cleaning.

Cell 22 · code · Raw numeric heatmap

#	Code	What it does
2-9	<code>raw_numeric = coords + passenger + target</code>	Columns that exist before engineering.
10	<code>.corr()</code>	Pearson matrix.
12-21	<code>sns.heatmap annot, cmap vlag, center 0</code>	Diverging colours around zero. Expect weak corr with duration.

D**Data cleaning**

Rule-based. No silent dropna. No IQR wipe of real airport runs.

Takes in
df_rawProduces
df — id and dropoff_datetime gone; impossible rows removed.**Cell 26 · code · apply_cleaning**

#	Code	What it does
1	<code>progress(...)</code>	Log Cluster D.
2	<code>def apply_cleaning(frame)</code>	Returns (cleaned_frame, report_table).
3-16	<code>nested log(step, before, after, note)</code>	NOT progress(). Appends one report row: rows in, rows out, % removed.
18-20	<code>copy; log 'loaded'</code>	Start from a copy of the argument.
22-24	<code>drop_duplicates()</code>	Exact duplicate rows (none expected).
26-28	<code>drop id, dropoff_datetime</code>	Label + leakage. Row count unchanged (column drop only).
30-37	<code>in_box = lon/lat between NYC bounds for pickup AND dropoff</code>	Bitwise AND of four .between() masks.
38-39	<code>out = out.loc[in_box]</code>	Keep only in-city GPS.
41-43	<code>passenger_count.between(1, 6)</code>	Drop 0 and 7+.
45-47	<code>trip_duration.between(60, 7200)</code>	Drop <1 min and >2 h.
49-51	<code>reset_index; return out, DataFrame(tracker)</code>	Clean index; report for the PDF.
54-57	<code>df, cleaning_report = apply_cleaning(df_raw); display; print retain %</code>	This is the table Cluster E receives.

E**Feature engineering**

Build distance and calendar columns. Never speed = distance / duration.

Takes in
df (cleaned)Produces
df_fe — modelling table. pickup_datetime replaced by
hour flags.**Cell 30 · code · manhattan_km, bearing_deg, engineer_features**

#	Code	What it does
2-5	manhattan_km	Convert $\Delta\text{lat}/\Delta\text{lon}$ to km (111.32 km per degree lat; lon shrinks by $\cos(\text{mean lat})$) and add the absolute legs — L1 / grid distance.
8-13	bearing_deg	Initial compass bearing pickup $\square$ dropoff, wrapped to 0–360°.
16-17	engineer_features: copy + to_datetime	Do not mutate the caller's frame.
18-21	lat1, lon1, lat2, lon2 as numpy	Fast vectorised geometry.
23-25	trip_distance_km, manhattan_distance_km, bearing_deg	The three spatial features.
26-28	hour, dayofweek, month as int	Calendar from pickup time. Monday=0 ... Sunday=6.
29	is_weekend = dayofweek >= 5	Saturday and Sunday.
30-31	is_rush_hour = weekday hours 7-9 or 16-19	Peak congestion flag.
32-33	drop pickup_datetime; return	Raw timestamp is no longer a column.
36	df_fe = engineer_features(df)	Official modelling table.
38-42	if FAST_DEV: sample 120k	Practice-run shortcut. Submission uses FAST_DEV False.
44-46	print new column names; df_fe.head()	Sanity check.

Cell 31 · code · Engineered heatmap

#	Code	What it does
2-14	engineered_cols list	Numerics we actually train on, plus the target.
15	corr_fe = df_fe[...].corr()	After cleaning, distance vs duration should be ~0.7+.
17-26	heatmap	Same style as Cluster C, now on real predictors.
28-29	print corr with target, sorted by r	Rank features by linear association.

Cell 33 · code · Cleaned distance scatter + rush-hour boxplot

#	Code	What it does
2	plot_fe = take_plot_sample(df_fe)	25k of the cleaned engineered table.
4-16	scatter distance vs duration	Should now be a tight upward band.
18-26	boxplot is_rush_hour	Same distance, worse time in peak. Relabel 0/1 as Off-peak / Rush hour.

F**Split, encode, scale**

Last pandas step. All ten models share these arrays.

Takes in
df_fe

Produces
X_train_p, X_test_p, y_train, y_test, preprocess,
feature_names

Cell 37 · code · 80/20 stratified split

#	Code	What it does
2-4	<code>y = duration; X = everything else</code>	Separate label from features. X still has vendor, coords, distances, flags.
6	<code>y_bins = pd.qcut(y, q=10, ...)</code>	Ten duration buckets. Regression's stand-in for class labels.
7-13	<code>train_test_split(..., stratify=y_bins, test_size=0.20, random_state=42)</code>	Long trips appear in both sides. Reproducible.
15-17	<code>print shapes and mean/median y</code>	Train and test should look similar. If not, stratification failed.

Cell 38 · code · ColumnTransformer + shared test cap

#	Code	What it does
2-3	<code>categorical vs numeric column lists</code>	vendor_id and store_and_fwd_flag are names, not ordered numbers.
5-14	<code>ColumnTransformer: StandardScaler on nums, OneHotEncoder on cats</code>	One object, two transformers. remainder='drop' discards anything forgotten.
16-17	<code>sparse_output=False, handle_unknown='ignore'</code>	Dense ndarray for all models; unseen categories become zeros instead of crashing.
19	<code>preprocess.fit_transform(X_train)</code>	CRITICAL: fit on train only. Means and dummy columns are learned here.
20	<code>preprocess.transform(X_test)</code>	Apply the same means/dummies. Never fit again.
21	<code>get_feature_names_out()</code>	num__trip_distance_km, cat__vendor_id_1, ... — names for coefficients and importances.
25-31	<code>if len(test) > 50_000: sample 50k with seed 42</code>	Same eval_idx for every later model. Fair comparison; SVR/KNN can predict in reasonable time.
32-33	<code>else: reset y_test index</code>	Align y with the matrix rows.
35-39	<code>print feature lists and matrix shapes</code>	Confirm width (numerics + dummy columns).

G**Ten regression models**Same `X_train_p` / `X_test_p`. Separate fit cell per algorithm.Takes in
Processed matrices from FProduces
leaderboard, predictions, `fitted_models`**Cell 41 · code · `evaluate()` and mean baseline**

#	Code	What it does
2-4	<code>leaderboard, predictions, fitted_models = empty</code>	Shared stores. Cluster I reads <code>fitted_models</code> .
7-14	<code>metrics_dict(...)</code>	One row: name, train rows, R^2 , RMSE, MAE, fit seconds.
17	<code>def evaluate(name, model, X_tr, y_tr, ...)</code>	Default test is the shared <code>X_test_p</code> / <code>y_test</code> from F.
18	<code>progress('fitting', ...)</code>	Log before the slow call.
19-20	<code>time; model.fit</code>	Train. This is the only <code>.fit</code> in the helper.
22	<code>y_pred = clip(predict(X_te), 1, None)</code>	No negative seconds. Clip is inference-only, not leakage.
23-24	<code>metrics + progress('done')</code>	Print R^2 / RMSE / MAE and how long fit took.
25-28	<code>append to leaderboard; save pred; save model</code>	<code>store=True</code> by default.
32-38	<code>Mean baseline = always y_train.mean()</code>	$R^2 \approx 0$ by construction. Floor every model must beat.

Cell 42 · code · `subset_train`

#	Code	What it does
1-6	<code>n = min(max_rows, len(X_train_p)); rng.choice without replacement</code>	Reproducible subset of processed training rows for SVR / KNN / forest search.
7-10	<code>iloc if DataFrame else numpy indexing</code>	<code>ColumnTransformer</code> returns a <code>ndarray</code> , so the numpy branch is the one that runs.

G1–G10 — each algorithm. Markdown above each cell states the idea. Code below is the fit.

Cell 45 · code · Linear Regression

#	Code	What it does
2-3	<code>LinearRegression(); evaluate on full X_train_p</code>	OLS, no penalty. Baseline.
5-9	<code>Series(lin.coef_, index=feature_names).sort by coef </code>	Because of <code>StandardScaler</code> , +200 on distance $\approx$ +200 seconds per +1 SD of distance.
11-12	<code>print intercept</code>	Predicted seconds when all scaled features are 0 (i.e. at the training mean).

Cell 48 · code · Ridge

#	Code	What it does
2-3	<code>Ridge(alpha=1.0); evaluate</code>	L2 penalty. Shrinks collinear distance weights instead of letting one go huge-positive and one negative.
5-8	<code>compare mean coef Linear vs Ridge</code>	Ridge should be slightly smaller on average.

Cell 50 · code · Lasso

#	Code	What it does
2-3	<code>Lasso(alpha=0.5, max_iter=20000)</code>	L1. <code>max_iter</code> high because coordinate descent can be slow.

#	Code	What it does
5-12	count coefficients with <code> w < 1e-10</code> ; display the rest	Sparsity check. Distance should survive.

Cell 52 · code · ElasticNet

#	Code	What it does
2-3	<code>ElasticNet(alpha=0.2, l1_ratio=0.5)</code>	Half L1, half L2. One evaluate call — no extra plots.

Cell 54 · code · Polynomial degree 1 vs 2

#	Code	What it does
2-3	<code>poly_comparison, poly_models</code>	Hold both degrees.
4-19	for degree in (1, 2): <code>Pipeline(PolynomialFeatures → LinearRegression)</code>	Degree 1 $\approx$ linear. Degree 2 adds squares and pairwise products (distance $\times$ hour). <code>include_bias=False</code> because <code>LinearRegression</code> has an intercept.
20-26	<code>clip predictions; record n_output_features_ and metrics</code>	Watch the column explosion at degree 2.
30-42	<code>pick the better R²; append as 'Polynomial Regression'</code>	One official leaderboard row, not two.

Cell 56 · code · Decision Tree

#	Code	What it does
2-6	<code>max_depth=12, min_samples_leaf=50, random_state=42</code>	Stops a giant tree from memorising GPS jitter.
7	<code>evaluate(...)</code>	Fit + score.
9-13	<code>feature_importances_ as a Series</code>	Fraction of MSE reduction. Should be led by distance.
15-20	<code>horizontal bar of top 12; finalise</code>	Rubric C4 tree importance plot.

Cell 59 · code · Random Forest

#	Code	What it does
2-9	<code>n_estimators=80, max_depth=18, min_samples_leaf=20, n_jobs=-1, verbose=1</code>	80 trees; all CPU cores; verbose prints tree progress during Run All.
10	<code>evaluate on full training matrix</code>	This is one of the slow cells.
12-16	<code>forest.feature_importances_</code>	More stable than a single tree's ranking.

Cell 61 · code · HistGradient Boosting

#	Code	What it does
2-9	<code>learning_rate=0.08, max_depth=6, max_iter=200, l2=0.1, verbose=1</code>	Sklearn's histogram GBM — same family as <code>GradientBoostingRegressor</code> , built for large n. verbose prints each iteration.
10	<code>evaluate(...)</code>	Usually the R^2 leader.

Cell 63 · code · SVR (12k subset)

#	Code	What it does
2-3	<code>X_svr, y_svr = subset_train(12_000)</code>	Fit is $O(n^2)$. Do not pass 279k rows.
5-8	<code>TransformedTargetRegressor(SVR(rbf, C=1, epsilon=0.1), StandardScaler())</code>	Scale y because C/epsilon assume small numbers; invert to seconds on predict.
9	<code>evaluate on the subset, still scoring the shared X_test_p</code>	Fair test set; smaller train set — see Train rows.

Cell 66 · code · KNN (40k subset)

#	Code	What it does
2-3	<code>subset_train(40_000)</code>	fit is instant (just stores rows). predict searches neighbours — that is why we cap n.
5	<code>KNeighborsRegressor(k=15, weights='distance', n_jobs=-1)</code>	Closer neighbours count more. Needs scaled X (already true).
6	<code>evaluate(...)</code>	Fit seconds ≈ 0 ; the cell time is mostly predict.

H**Compare, tune, check**

One table. Search Ridge and Forest. Residuals. 5-fold CV of the top two official models.

Takes in
leaderboard + fitted models

Produces
Ranked table, tuned models, residual plots, cv_rows

Cell 69 · code · Leaderboard + R² bar chart

#	Code	What it does
2-6	<code>DataFrame(leaderboard).sort_values R2 descending; Rank = index+1</code>	Rubric C2: one table, same test trips.
7-14	<code>style.format R2 4dp, RMSE/MAE 1dp</code>	Readable numbers.
16-20	<code>barh of R², drop the mean baseline from the bars</code>	Visual ranking, finalise saves the PNG.

Cell 73 · code · GridSearchCV on Ridge

#	Code	What it does
2-9	<code>GridSearchCV(Ridge, alpha grid, scoring='r2', cv=5, n_jobs=-1, verbose=2)</code>	Tries each alpha with 5-fold CV on TRAIN. verbose prints each candidate.
10-12	<code>fit; print best_params_ and best_score_</code>	<code>best_score_</code> is CV R ² , not test R ² .
14	<code>look up untuned Ridge test R² from leaderboard</code>	For the before/after sentence.
15-20	<code>evaluate Ridge(**best_params_) as 'Ridge Regression (tuned)'</code>	Refit on all training rows, score the shared test set.
21-23	<code>print test R² before vs after and the delta</code>	Rubric C3: show the improvement (may be ~0 if alpha=1 was already best).

Cell 74 · code · RandomizedSearchCV on Forest, then full refit

#	Code	What it does
2	<code>X_tune, y_tune = subset_train(40_000)</code>	Search on a subset so 8×3 CV finishes.
4-16	<code>RandomizedSearchCV: n_estimators, max_depth, min_samples_leaf, max_features; n_iter=8, cv=3, verbose=2</code>	Random 8 draws, not a full grid.
17-19	<code>fit on subset; print best params + subset CV R²</code>	These params are not yet the test score.
21-27	<code>evaluate RandomForestRegressor(**best_params_) on FULL X_train_p</code>	The fair test number. <code>n_jobs=-1</code> again.
28-30	<code>print test R² before vs after</code>	Document the gain.

Cell 75 · code · Leaderboard including tuned rows

#	Code	What it does
1-10	<code>rebuild results_tuned from leaderboard, same formatting</code>	Tuned Ridge / Forest now appear as extra rows. Official ten are still the untitled names.

Cell 78 · code · Predicted vs actual + residuals

#	Code	What it does
2-13	<code>official = the ten required names</code>	CV later uses this list.

#	Code	What it does
15-19	<code>candidates = official plus tuned Ridge/Forest; best_name = max R²</code>	Best for diagnostics may be a tuned sibling.
20-21	<code>y_best = predictions[best_name]; residuals = y_test - y_best</code>	Residual = actual – predicted, in seconds.
26-34	<code>left scatter: predicted vs actual + y=x line</code>	A good model hugs the diagonal on typical trips.
36-40	<code>right scatter: residual vs predicted + y=0 line</code>	Funnel = worse on long trips (no traffic feature).
42-43	<code>finalise; print residual mean and std</code>	Mean near 0 = unbiased on average.

Cell 79 · code · Forest feature importance

#	Code	What it does
2-3	<code>use forest_tuned if it exists, else forest</code>	<code>dir()</code> check after the search cell has run.
4-8	<code>top 12 importances, smallest at the top of a barh</code>	Distance should dominate.

Cell 83 · code · 5-fold CV of the two best official models

#	Code	What it does
2-7	<code>ranked_official = leaderboard query Model in @official; take head 2</code>	Ignores tuned extra rows and the mean baseline. Uses pandas query with the official list.
10-37	<code>rebuilders dict of lambdas</code>	Fresh unfitted copies with the same hyperparameters as Cluster G (not the search winners). Needed because <code>cross_val_score</code> refits.
39-47	<code>for each of the two: pick X_cv (full train, or SVR/KNN subset)</code>	Test set stays untouched.
48-49	<code>progress('CV'); cross_val_score(..., cv=5, scoring='r2', n_jobs=-1)</code>	Five R ² numbers on training folds.
50-60	<code>store mean, std, fold list; print; display table</code>	Stability: small std is good.

I**Manual tester**

Type a trip. Same engineer_features + already-fitted preprocess. No refit.

Takes in
fitted_models, preprocess, X_train.columnsProduces
predicted seconds / minutes for your dictionaries**Cell 86 · code · manual_trip dictionaries + predict_manual_trips**

#	Code	What it does
6-15	<code>manual_trip = {vendor, datetime, passengers, 4 coords, flag}</code>	EDIT THESE VALUES. Example is a weekday 17:24 Manhattan hop (rush hour).
18-27	<code>manual_trip_b = late-night trip (or set None to skip)</code>	Second sanity check: similar distance at 00:43 should predict shorter.
30-32	<code>predict_manual_trips(*trips): drop Nones; DataFrame</code>	One row per trip.
33-45	needed columns list; raise if missing	Must match what engineer_features expects (cleaned-table columns, including pickup_datetime).
47	<code>fe = engineer_features(raw)</code>	Same function as Cluster E — distance, hour, rush-hour, etc.
48-50	reindex to X_train.columns; error on NaN	Column order/names must match training. A typo in the dict shows up here.
51	<code>X_manual_p = preprocess.transform(X_manual)</code>	TRAIN scaler/encoder — never fit. This is the leakage-safe path.
54-57	best = top R ² name that exists in fitted_models	Usually HistGBM or a tuned forest.
59-63	predict, clip at 1s, also minutes	Human-readable duration.
66-75	display trip table (distance, hour, rush-hour, prediction)	Check that engineering looks right before trusting the number.
78-86	for every fitted model, predict trip 1	Viva table: trees vs linear on the same handmade ride.
91	<code>manual_predictions, manual_model_compare = predict_manual_trips(...)</code>	Saved for Cluster J's PDF page.

After Run All, you only need to re-run this cell when you change the numbers. Do not refit Cluster G.

J**PDF report**

Pack live tables and every PNG from finalise() into results/regression_report.pdf.

Takes in
 SAVED_FIGURES, leaderboard, cleaning_report,
 cv_rows, manual_predictions

Produces
 results/regression_report.pdf

Cell 89 · code · PdfPages writer

#	Code	What it does
1	PDF_PATH = RESULTS_DIR / 'regression_report.pdf'	Output file.
2-6	leaderboard_df ranked + Rank column	Same metrics table as Cluster H.
7	cv_df from cv_rows if the CV cell ran	Guard with dir() so a partial run does not crash the export.
10-16	_page_text(pdf, title, lines)	Title page: row counts, FAST_DEV, best model. axis off, savefig into the PdfPages.
19-36	_page_table(pdf, title, frame)	matplotlib table of a DataFrame (cleaning, metrics, CV, manual).
39-50	_page_image(pdf, path)	Embed a saved PNG, preserve aspect, one figure per page.
53-end	with PdfPages(...) as pdf:	Cover ☐ cleaning ☐ metrics ☐ CV ☐ manual tester ☐ each SAVED_FIGURES PNG in order.
last prints	print path and figure count	Confirm the file landed.

This Cluster J PDF is the results pack (plots + numbers). The file you are reading now is the code guide (what each cell means).

Viva cheat-sheet

Why drop dropoff_datetime? Duration is dropoff minus pickup. Using it is cheating (leakage).

Why fit the scaler on train only? Test means would leak future information into the scale. Marks off.

Why is Manhattan distance's linear coefficient sometimes negative? It is collinear with haversine. OLS splits credit. Ridge/Lasso exist for that. Trees just pick one.

Why does KNN fit in 0.0 s but only use 40k rows? Fit only stores rows. Predict searches neighbours. Cost is train $\times$ test $\times$ features.

Why SVR on 12k? RBF SVR training is roughly quadratic to cubic in n.

Why HistGradientBoosting instead of GradientBoostingRegressor? Same sklearn boosting idea; histogram version is the one that finishes on hundreds of thousands of rows.

What must a serious model beat? The mean baseline, $R^2 \approx 0$, RMSE ≈ 10 –11 minutes on this table.

Run order if something breaks: A ☐ B ☐ C ☐ D ☐ E ☐ F ☐ G ☐ H ☐ I ☐ J. Do not jump to G without F. Do not jump to I without G.

Generated from the cell order of notebooks/regression.ipynb. Course 23CSE301 Machine Learning, Review 1, regression track.